

p0945r0

Generalizing alias declarations

Published Proposal, 10 February 2018

This version:

<http://wg21.link/p0945r0>

Author:

[Richard Smith](#) (Google)

Audience:

EWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Abstract

This paper proposes extending *alias-declarations* to allow the formation of aliases for all names, not only type names.

Table of Contents

1	Problem
2	Design principles
3	Proposal
3.1	Member aliases
3.2	Templated aliases
4	Interactions with other proposals
5	Acknowledgements

References

Informative References

§ 1. Problem

Large-scale interface refactorings often follow a three-step process:

- Change the interface so both the old version and the new version are concurrently available
- Change all users of the old version to use the new version (for externally-released libraries, marking the old version as `[[deprecated]]` is a useful tool for this)
- Remove the old version

C++ provides some language features to help out with the first step of this process. *using-declarations* provide a tool to allow a name to be exposed in two different namespaces. *alias-declarations* and `typedef` declarations provide a tool to allow a type to be exposed with two different names. Function overloading provides a tool to allow a function to present two different signatures. However, C++ lacks a general mechanism to allow an entity to be exposed with two different names.

There are a set of workarounds for this problem, for renaming different kinds of entities:

- A type can be renamed using an *alias-declaration*.
- A function (or function template) can be renamed by providing a forwarding function. However, it is not possible to perfectly forward parameters passed by value, so this may introduce additional moves of function parameters. Also, the functions can be distinguished by address, although this seldom matters.
- A variable (or variable template) can be renamed by providing a (`constexpr inline`) reference variable: `constexpr inline auto &old_name = new_name;`
- A non-static data member cannot be renamed without occupying additional storage within the object (for instance, by adding a reference member bound to another member; this also prevents the structure from being trivially-copyable).
- An enumerator can be renamed by using a `constexpr inline` variable.
- A namespace can be renamed by using a *namespace-alias* declaration: `namespace old_name = new_name;`
- A type template can be renamed by using an alias template (but the default arguments must be duplicated, and the alias is distinguishable from the original when used as a template template argument).

- A concept can be renamed by defining another concept in terms of it: `template<typename T>
concept OldName = NewName<T>;`

However, aside from the special-case *alias-declaration* and *namespace-alias* syntaxes, none of these directly represents the intent of simply binding another name to an entity.

§ 2. Design principles

Note: These principles are copied from Herb's [\[P0515R3\]](#) at the encouragement of that paper.
Thanks, Herb!

The primary design goal is [conceptual integrity](#), which means that the design is coherent and reliably does what the user expects it to do. Conceptual integrity's major supporting principles are:

- *Be consistent:* Don't make similar things different, including in spelling, behavior, or capability. Don't make different things appear similar when they have different behavior or capability. -- For example, this paper follows this principle in using the exact same syntax to imbue an entity with a new name, regardless of the kind of entity, and semantic differences between the newly-proposed feature and the pre-existing *using-declaration* are removed.
- *Be orthogonal:* Avoid arbitrary coupling. Let features be used freely in combination. -- For example, this paper permits templated *alias-declarations*.
- *Be general:* Don't restrict what is inherent. Don't arbitrarily restrict a complete set of uses. Avoid special cases and partial features. -- For example, this paper permits arbitrary renaming between members and non-members, except in cases where the result would be meaningless or problematic (such as a namespace-scope class member, or a class-scope namespace name).

§ 3. Proposal

This paper proposes to extend *alias-declarations* to permit aliasing any entity. This is not a novel idea; the original paper proposing alias templates ([\[N1449\]](#)) described this as an extension of the base functionality, and indeed "[the] motivation for using any keyword at all [for *alias-declarations*] stemmed partly from the desire to use a syntax that might be compatible with [aliasing more general entities]".

The specific syntax proposed is:

alias-declaration:

```
using identifier attribute-specifier-seqopt = id-expression ;
```

The *identifier* is declared as a name for the entity or overload set named by the *id-expression*. The program is ill-formed if this results in a conflicting meaning for the name within its declarative region, but this syntax may be used to redeclare a name to refer to the same entity to which it already referred.

The resulting declared identifier is declared as the same kind of name as the *id-expression*: it is a *class-name* if the *id-expression* was a *class-name*, is a *namespace-name* if the *id-expression* was a *namespace-name*, is a *typedef-name* if the *id-expression* was a *typedef-name*, and so on.

As with the existing *alias-declaration*, this new form is permitted at namespace scope, block scope, and within class definitions. An *alias-declaration* that names a non-static data member or non-static member function may only appear as a *member-declaration* of the same class or one of its derived classes. An *alias-declaration* that names a namespace may not appear at class scope. No other restrictions are proposed on the kind of entity that may be named by the *id-expression*.

For consistency, we propose removing those restrictions from *using-declarations* that are not present for this form of *alias-declaration*:

- A class-scope *using-declaration* may name a non-class member
- A non-class-scope *using-declaration* may name a class member other than a non-static data member or non-static member function
- A *using-declaration* may name a namespace or a scoped enumerator

The general principle is to make the following declarations equivalent, so the former is merely a shorthand for the latter:

```
using A::B;  
using B = A::B;
```

We further propose deprecating *namespace-alias* declarations, as the more general *alias-declaration* syntax can be used in its stead.

§ 3.1. Member aliases

When an *alias-declaration* or *using-declaration* at class scope names a non-member, that non-member may be accessed by explicit qualification of the class name (`Class::Member`). If the name does not name a type nor a type template, class member access syntax may also be used (`instance.member`).

Non-member functions and variables brought into a class by such a declaration behave analogously to static class members:

```
void f(int a, int b);
struct Y {};
void g(Y);

struct X {
    using g = f;
    void h(Y y) {
        g(1, 2); // OK, calls f(1, 2)
        g(y);    // error, lookup finds X::g (== ::f)
    }
};

void h(X x) {
    x.g(1, 2); // OK, same as X::g(1, 2), same as ::f(1, 2)
}
```

Member *alias-declarations* or member *using-declarations* introduce member names of the enclosing class, but do not make the nominated entities members of the class. In particular, the nominated entities have no special access to non-public members of the class.

Member *alias-declarations* can be used to provide aliases for non-static data members and non-static member functions. For example:

```
template<typename K, typename V>
struct map<K, V>::value_type : public std::pair<const K, V> {
    using pair::pair;
    using key = first;
    using value = second;
};
```

For consistency with class-scope *using-declarations*, functions introduced by class-scope *alias-declarations* are hidden by declared class members with the same signature:

```

struct B {
    void f(int);
    void f(int, int);
};
struct C : B {
    using B::f;
    void f(int); // hides B::f(int) but not B::f(int, int)
};
struct D : B {
    using g = B::f;
    void g(int); // hides B::f(int) but not B::f(int, int)
};

```

An aliased virtual function may be overridden using either its original name or its aliased name. As usual, if there are multiple final overriders for a single virtual function, the program is ill-formed. Example:

```

struct B1 {
    virtual void f();
    using f1 = f;
};
struct B2 {
    virtual void f();
    using f2 = f;
};
struct D : B1, B2 {
    void f1() override; // OK, overrides B1::f but not B2::f
    void f() override; // error, multiple final overriders for B1::f in D
};

```

§ 3.2. Templated aliases

For consistency with existing *alias-declarations* and for maximal flexibility, we propose to allow the generalized *alias-declaration* to also be templated. For example, this permits a static data member of a class template to be renamed to a non-class-member template:

```
template<typename T> struct X { static int y; };
template<typename T> using Y = X<T>::y;
static_assert(&Y<int> == &X<int>::y);
```

... and permits an alias to reorganize the parameter list of a template, for instance if a new mandatory template parameter is needed:

```
template<typename T, typename U> pair<T, U> my_pair;
template<typename T> using old_pair = my_pair<T, int>;
```

For the purpose of template argument deduction, when an alias template names a function template specialization, the template arguments from the alias are first substituted into the function template(s) to form another set of function templates prior to deduction:

```
namespace New {
    template<typename Iterator> void process(Iterator begin, Iterator end);
}
namespace Old {
    // Substitutes Iterator = T* into New::process, forming
    //   template<typename T> void process(T *begin, T *end);
    template<typename T> using process = New::process<T*>;
}
void f() {
    int arr[5];
    Old::process(std::begin(arr), std::end(arr)); // ok, deduces T = int, so
}
```

When a templated *alias-declaration* has a dependent *nested-name-specifier*, it is not possible in general to determine whether it names a function (or set of overloaded functions), nor to perform template argument deduction against it; such a declaration is therefore not permitted to form part of an overload set and must instead be the only declaration with its name in its declarative region.

As with existing alias templates, the generalized form cannot be explicitly or partially specialized nor explicitly instantiated, and instead acts as a transparent alias for its (substituted) *id-expression*. The *id-expression* in a templated *alias-declaration* shall not name a template.

§ 4. Interactions with other proposals

[P0634R1] proposes that the `typename` keyword be made optional in a number of contexts. One of these is the right-hand side of an *alias-declaration*. That portion of that proposal conflicts with this proposal; in a case such as

```
template<typename T> struct X {  
    using A = T::something;  
    void f() { A * a; }  
};
```

... correctly parsing the definition of `X<T>::f()` requires knowledge of whether `A` is a type. Therefore we still require the `typename` keyword in dependent *alias-declarations* if we choose to move forward with both proposals.

§ 5. Acknowledgements

Thanks to Gabriel Dos Reis and Mat Marcus for proposing *alias-declarations* in [N1449]. Thanks to Jorg Brown for providing the `map::value_type` example, and to Matt Calabrese for providing early feedback on this proposal.

§ References

§ Informative References

[N1449]

G. Dos ReisM. Marcus. [Proposal to add template aliases to C++](https://wg21.link/n1449). URL: <https://wg21.link/n1449>

[P0515R3]

Herb Sutter, Jens Maurer, Walter E. Brown. [Consistent comparison](https://wg21.link/p0515r3). URL: <https://wg21.link/p0515r3>

[P0634R1]

Daveed Vandevoorde, Nina Ranns. [Down with `typename`!](https://wg21.link/p0634r1). URL: <https://wg21.link/p0634r1>